

Rapport de Projet

Marketing Mix Modeling et attribution Multi_Tach

Modélisation Bayésienne · Pipeline Data · Visualisation BI

Réalisé par :

Hamza Elmourabit · Saad Benhaimer

Encadré par : Pr.Hrimech

**Pipeline ETL · Feature Engineering · Modèle Bayésien ·
Dashboard Streamlit · Looker BI**

Résumé exécutif

Ce document constitue le rapport technique de référence du projet de **Marketing Mix Modeling (MMM)**. Il détaille l'ensemble de la chaîne de valeur, depuis l'ingestion des données brutes jusqu'à la restitution opérationnelle via des tableaux de bord interactifs.

Le projet repose sur quatre piliers :

1. **Pipeline Data** — validation, nettoyage, enrichissement événementiel, feature engineering (ad-stock, saturation, lags, interactions) et normalisation ;
2. **Modélisation Bayésienne** — estimation des contributions canal, élasticités et simulation budgétaire avec pymc et arviz ;
3. **Visualisation** — dashboard Streamlit multi-pages et intégration de dashboards Looker embarqués ;
4. **Industrialisation** — export BigQuery, schéma analytique, configuration centralisée et exécution automatisée.

Le système produit un jeu de données de **52+ variables** prêtes pour la modélisation, incluant les transformations MMM critiques (adstock géométrique, saturation de Hill, lags temporels et interactions inter-canaux), ainsi que les variables de contrôle (saisonnalité, événements, tendance).

Table des matières

Résumé exécutif	1
1 Introduction	4
1.1 Contexte.....	4
1.2 Objectifs du projet	4
1.3 Périmètre.....	4
2 Architecture du système	4
2.1 Vue d'ensemble.....	4
2.2 Composants détaillés	4
3 Pipeline de données	5
3.1 Flux de transformation	5
3.2 Étape 1 — Validation robuste.....	5
3.3 Étape 2 — Nettoyage intelligent.....	5
3.4 Étape 3 — Feature Engineering avancé	6
3.4.1 Adstock géométrique.....	6
3.4.2 Saturation — Transformation de Hill	6
3.4.3 Lags temporels.....	7
3.4.4 Interactions inter-canaux.....	7
3.4.5 Résumé des features créées	7
3.5 Étape 4 — Normalisation.....	8
3.6 Étape 5 — Préparation pour la modélisation	8
3.7 Enrichissement événementiel.....	8
3.8 Schéma BigQuery	8
4 Modélisation MMM	9
4.1 Objectifs.....	9
4.2 Approche Bayésienne.....	9
4.3 Spécification du modèle.....	9
4.4 Flux de modélisation	10
4.5 Diagnostics du modèle.....	10
4.6 Attribution multi-touch	10
5 Référentiel des variables	11
5.1 Variable cible	11
5.2 Canaux marketing.....	11
5.3 Variables de contrôle.....	11
6 Visualisation et BI	11
6.1 Architecture de visualisation	12
6.2 Dashboard Streamlit	12
6.2.1 Scénarios budgétaires (What-If).....	12
6.3 Intégration Looker.....	13
7 Déploiement	13
7.1 Prérequis	13
7.2 Installation	13
7.3 Variables de configuration	14
7.4 Exécution.....	14
7.5 Artefacts de production.....	14

8 Résultats attendus	14
8.1 Indicateurs de performance	15
9 Perspectives et développements futurs	15
10 Conclusion	15
A Structure des répertoires	16
B Exemple de code PyMC	16
C Modèles LookML	17

1 Introduction

1.1 Contexte

Dans un environnement marketing de plus en plus fragmenté, la mesure de l'efficacité publicitaire est un enjeu stratégique. Les méthodes d'attribution *last-click* ou *multi-touch* basées sur les cookies sont remises en question par les restrictions de confidentialité (iOS 14+, fin des tiers cookies). Le **Marketing Mix Modeling** offre une alternative robuste, agrégée et respectueuse de la vie privée.

1.2 Objectifs du projet

Le projet vise à délivrer une solution *end-to-end* permettant de :

- transformer les données brutes marketing et ventes en features exploitables pour la modélisation ;
- estimer la contribution marginale de chaque canal publicitaire au chiffre d'affaires ;
- simuler des allocations budgétaires optimisées ;
- fournir des interfaces de visualisation opérationnelles (Streamlit + Looker).

1.3 Périmètre

Le périmètre couvre 9 canaux marketing (Google Search, Shopping, PMax, Display, Video, Meta Facebook, Instagram, Other, TikTok), les revenus de premières ventes comme variable cible, et un enrichissement événementiel de 17+ jours spéciaux (Black Friday, Noël, etc.).

2 Architecture du système

L'architecture est organisée en **trois couches** : ingestion et transformation, modélisation, visualisation et BI.

2.1 Vue d'ensemble

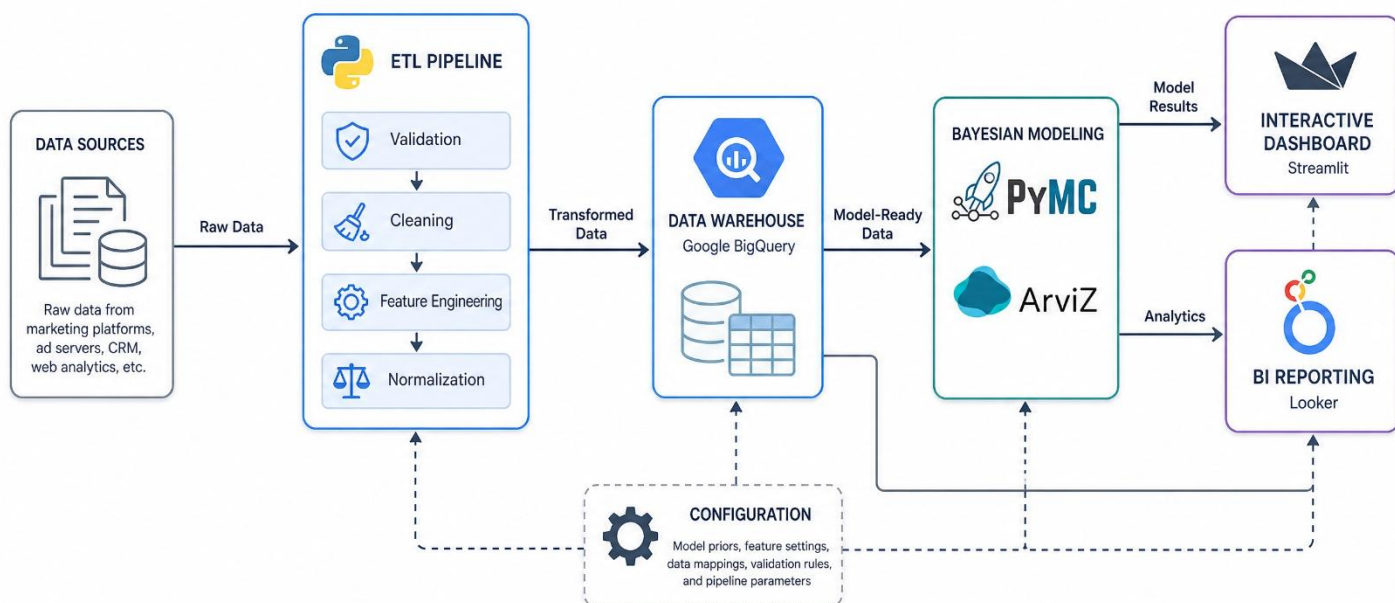


FIGURE 1 – Architecture globale du système MMM

2.2 Composants détaillés

Le tableau ci-dessous récapitule chaque composant, son rôle et son fichier principal.

Composant	Rôle	Fichier
Source de données	Fichiers marketing et ventes bruts	data/raw/
Validation	Vérification qualité (8 checks automatiques)	etl/validation.py
Nettoyage	Parse dates, NaN intelligents, agrégation	etl/clean.py
Enrichissement	Jours fériés, événements commerciaux	etl/event_enrichment.py
Feature Engineering	Adstock, saturation, lags, interactions	etl/feature_engineering.py
Normalisation	Z-score, log-transform, sauvegarde params	etl/normalize.py
Orchestration	Pipeline 5 étapes automatisée	etl/pipeline.py
BigQuery	Export multi-tables + vues analytiques	etl/load_bigquery.py
Configuration	Paramètres centralisés (decay, saturation, paths)	etl/config.py
Modélisation	Modèle Bayésien MMM (PyMC + ArviZ)	models/
Dashboard	App Streamlit multi-pages	dashboard/app.py
Looker	Embed Looker + templates LookML	dashboard/looker_integration.py

TABLE 1 – Récapitulatif des composants du système

3 Pipeline de données

La pipeline ETL constitue le cœur du projet. Elle transforme les données brutes en un jeu de données analytiquement exploitable, en 5 étapes orchestrées.

3.1 Flux de transformation

```
PS C:\Users\khadi\MMM_Project\MMM_Project> # Option 1: Pipeline complète en une ligne
PS C:\Users\khadi\MMM_Project\MMM_Project> python run_pipeline.py
```

```
=====
🚀 MARKETING MIX MODELING - DATA PIPELINE
=====

=====
🚀 MARKETING MIX MODELING - ETL PIPELINE COMPLÈTE
=====

[1/5] VALIDATION DES DONNÉES BRUTES
-----
📁 Chargement: C:\Users\khadi\MMM_Project\MMM_Project\data\raw\compressed_data.csv
✅ 132759 lignes, 50 colonnes chargées
🔍 Validation des données en cours...
⚠️ 2533862 valeurs manquantes détectées
ORGANISATION_VERTICAL            8110
ORGANISATION_SUBVERTICAL         8110
GOOGLE_PAID_SEARCH_SPEND         36122
GOOGLE_SHOPPING_SPEND            53779
GOOGLE_PMAX_SPEND                71633
GOOGLE_DISPLAY_SPEND             114013
GOOGLE_VIDEO_SPEND               125131
META_FACEBOOK_SPEND              52647
META_INSTAGRAM_SPEND             105942
META_OTHER_SPEND                 109214
```

FIGURE 2 – Flux de la pipeline ETL en 5 étapes

3.2 Étape 1 — Validation robuste

Validation — DataValidator

Le module `etl/validation.py` applique 8 vérifications automatiques sur les données brutes :

1. Valeurs manquantes et pourcentage par colonne
2. Format des dates (détection de dates invalides)
3. Plages numériques (pas de dépenses négatives impossibles)
4. Doublons (par date + vertical + sub-vertical)
5. Types de données corrects
6. Cohérence du ratio spend/revenue
7. Cardinalité des colonnes catégorielles
8. Détection d'outliers extrêmes ($> 5\sigma$)

3.3 Étape 2 — Nettoyage intelligent

Le module `etl/clean.py` remplace le `fillna(0)` naïf par un traitement différencié :

Type de colonne	Stratégie	Justification
Dépenses (spend)	fillna(0)	L'absence de spend signifie aucune dépense ce jour
Métriques (revenue)	Interpolation linéaire puis médiane	Conserve la tendance sans biais
Dates invalides	Suppression ou correction manuelle	Date absente = ligne inutilisable
Doublons	Agrégation par somme	Conserve le volume total
Valeurs négatives	Correction à 0 + log warning	Spend négatif = erreur de collecte

TABLE 2 – Stratégies de nettoyage par type de variable

Le nettoyage produit également :

- des **canaux agrégés** (GOOGLE_TOTAL_SPEND, META_TOTAL_SPEND) ;
- des **métriques dérivées** (ROI, CPA par canal).

3.4 Étape 3 – Feature Engineering avancé

C'est l'étape **critique** pour le MMM. Le module `etl/feature_engineering.py` crée plus de 45 colonnes transformées à partir des 9 canaux bruts.

3.4.1 Adstock géométrique

Transformation d'Adstock

L'adstock modélise l'effet de mémoire des investissements publicitaires :

$$\text{adstock}(x_t; \lambda) = x_t + \lambda x_{t-1} + \lambda^2 x_{t-2} + \dots + \lambda^{W-1} x_{t-W+1} = \sum_{w=0}^{W-1} \lambda^w x_{t-w} \quad (1)$$

où $\lambda \in [0, 1]$ est le **taux de décroissance** (decay) et W est la fenêtre de rémanence (par défaut 13 jours).

Canal	Decay λ	Demi-vie (j)	Interprétation
Google Paid Search	0.50	≈ 1.3	Intent rapide, court terme
Google Shopping	0.40	≈ 0.9	Très immédiat, e-commerce
Google PMax	0.60	≈ 1.7	Moyen terme, multi-format
Google Display	0.70	≈ 2.3	Awareness, long terme
Google Video	0.65	≈ 1.9	Notoriété, moyen-long terme
Meta Facebook	0.60	≈ 1.7	Engagement moyen terme
Meta Instagram	0.65	≈ 1.9	Storytelling visuel
Meta Other	0.55	≈ 1.5	Divers social
TikTok	0.70	≈ 2.3	Viralité, portée longue

TABLE 3 – Paramètres d'adstock par canal marketing

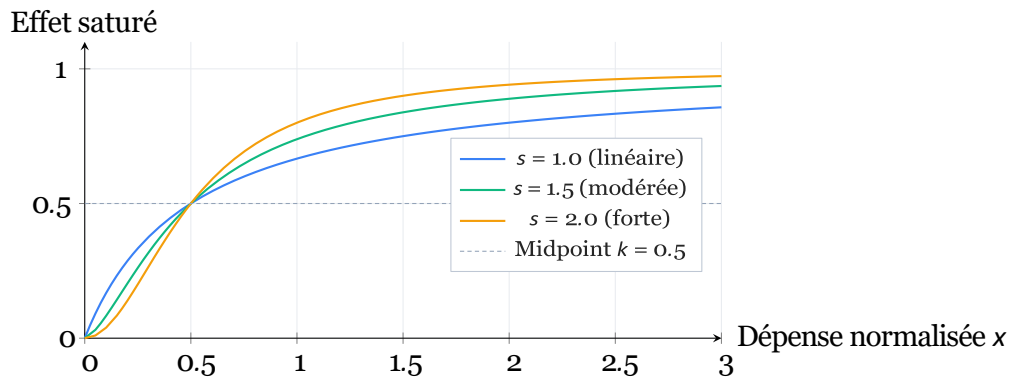
3.4.2 Saturation – Transformation de Hill

Transformation de Hill

La saturation modélise les rendements décroissants :

$$\text{sat}(x; s, k) = \frac{x^s}{k^s + x^s} \quad (2)$$

où $s > 0$ contrôle la pente (vitesse de saturation) et $k > 0$ est le point d'inflexion (midpoint, où l'effet atteint 50 % du maximum).

FIGURE 3 – Courbes de saturation de Hill pour différentes valeurs de s ($k = 0.5$)

3.4.3 Lags temporels

Des décalages temporels sont créés pour capturer les effets retardés :

- **Lag-1** : impact à $t - 1$ (effet immédiat + decay) ;
- **Lag-2** : impact à $t - 2$ (effet moyen terme) ;
- **Lag-4** : impact à $t - 4$ (effet court-moyen terme).

Cela génère $3 \times 9 = 27$ colonnes *_LAG1/2/4.

3.4.4 Interactions inter-canaux

Deux interactions clés sont construites pour capturer les synergies :

$$\text{GOOGLE_META_INTERACTION} = \text{GOOGLE_ADSTOCK_SAT} \times \text{META_ADSTOCK_SAT} \quad (3)$$

$$\text{SEARCH_DISPLAY_INTERACTION} = \text{SEARCH_ADSTOCK_SAT} \times \text{DISPLAY_ADSTOCK_SAT} \quad (4)$$

Interprétation des interactions

- L'interaction (3) capte la **synergie digitale** : un utilisateur exposé à Google ET Meta a un effet combiné supérieur à la somme des effets isolés.
- L'interaction (4) capte l'effet de **brand building** : le Display crée la notoriété, le Search convertit.

3.4.5 Résumé des features créées

Transformation	Nb colonnes	Suffixe
Adstock géométrique	9	*_ADSTOCK
Adstock + Saturation	9	*_ADSTOCK_SAT
Lags (1, 2, 4 jours)	27	*_LAG1/2/4
Interactions	2	GOOGLE_META_INTERACTION, ...
Variables de contrôle	5+	tendance, sin/cos mois, événements
Total	52+	

TABLE 4 – Récapitulatif des features générées par le Feature Engineering

3.5 Étape 4 — Normalisation

Standardisation Z-score

Les variables sont centrées-réduites pour favoriser la convergence du sampler Bayésien :

$$\tilde{x} = \frac{x - \mu}{\sigma} \quad (5)$$

Les paramètres μ et σ sont sauvegardés dans `normalization_params.pkl` pour permettre la dénormalisation inverse des prédictions.

Les dépenses asymétriques subissent une **log-transformation** préalable : $x_{\log} = \ln(1 + x)$.

3.6 Étape 5 — Préparation pour la modélisation

L'ultime étape :

- supprime les lignes avec `revenue=0` (pas d'information pour le modèle) ;
- sélectionne les colonnes pertinentes (52+) ;
- exporte `mmm_ready.csv` et charge dans BigQuery.

3.7 Enrichissement événementiel

Le module `etl/event_enrichment.py` ajoute des flags pour les jours spéciaux, classés par niveau d'impact :

Niveau	Flag	Exemples
CRITICAL	<code>is_critical_event=1</code>	Black Friday, Cyber Monday
HIGH	<code>is_high_impact=1</code>	Noël, Thanksgiving, Pâques
MEDIUM	<code>is_medium_impact=1</code>	Saint-Valentin, événements commerciaux
LOW	<code>is_holiday=1</code>	Jour du travail, changement d'heure

TABLE 5 – Classification des événements par impact

3.8 Schéma BigQuery

La table BigQuery est partitionnée par date et structurée en 9 sections :

Section	Contenu
1	Identifiants (date, vertical, territory)
2	Revenue (variable cible)
3	Dépenses brutes (9 canaux)
4	Adstock (9 colonnes)
5	Saturation (9 colonnes)
6	Lags (27 colonnes)
7	Interactions (2 colonnes)
8	Variables de contrôle (tendance, cyclicité)
9	Métriques dérivées (ROI, CPA)

TABLE 6 – Sections du schéma BigQuery

Trois vues analytiques sont créées automatiquement : `mmm_by_channel`, `mmm_daily_roi`, `mmm_by_vertical`.

4 Modélisation MMM

4.1 Objectifs

La modélisation vise à :

1. estimer la **contribution marginale** de chaque canal ;
2. mesurer les **élasticités** de dépense ;
3. permettre des **simulations budgétaires** (what-if) ;
4. fournir une base pour l'**attribution multi-touch**.

4.2 Approche Bayésienne

Le projet adopte une approche Bayésienne pour plusieurs raisons :

- **Incorporation de connaissances métier** via les distributions *a priori* (priors) ;
- **Quantification des incertitudes** — distributions postérieures au lieu de coefficients ponctuels ;
- **Régularisation naturelle** — les priors contraignent les coefficients dans des plages réalistes ;
- **Diagnostics de convergence** avec ArviZ ($\hat{R}$, ESS, trace plots).

4.3 Spécification du modèle

Modèle Bayésien MMM

Priors : $\alpha \sim N(0, 2)$

(intercept)

(6)

$\theta_c \sim N^+(0, 1)$

coefficients canal, $c =$

(7)

$\theta_\ell \sim N(0, 1)$

coefficients lags, $\ell =$

(8)

$\gamma_{\text{int}} \sim N(0, 1)$

interactions (2)

(9)

$\gamma_{\text{sais}} \sim N(0, 1)$

saisonnalité (sin/cos)

(10)

$\gamma_{\text{evt}} \sim N(0, 1)$

événements

(11)

$\gamma_{\text{trend}} \sim N(0, 1)$

tendance

(12)

$\sigma \sim \text{HalfNormal}(1)$

bruit

(13)

$$\text{Likelihood : } \mu_t = \alpha + \sum_{c=1}^9 \theta_c \tilde{x}_{c,t}^{\text{sat}} + \sum_{\ell} \theta_{\ell} \tilde{x}_{\ell,t} + \gamma_{\text{int}}^T \mathbf{z}_{\text{int } t}^{\text{int}} + \gamma_{\text{sais}}^T \mathbf{z}_{\text{sais } t}^{\text{sais}} + \gamma_{\text{evt}}^T \mathbf{z}_{\text{evt } t}^{\text{evt}} + \gamma_{\text{trend}} t \quad (14)$$

$$y_t \sim N(\mu_t, \sigma^2) \quad (15)$$

où $\tilde{x}_{c,t}^{\text{sat}}$ désigne le spend du canal c après adstock + saturation (normalisé).

4.4 Flux de modélisation

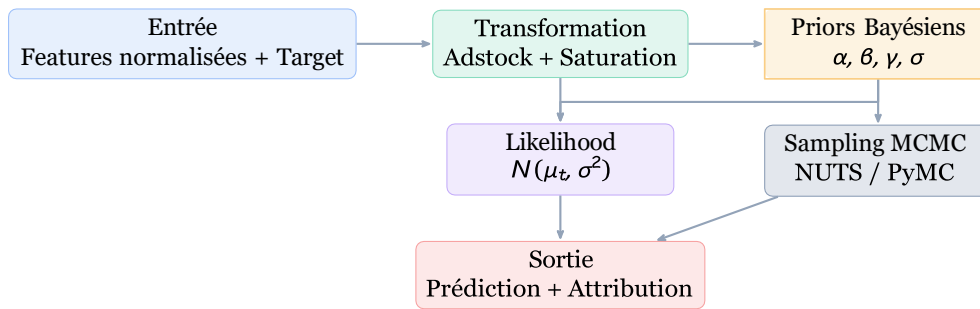


FIGURE 4 – Flux de modélisation Bayésienne MMM

4.5 Diagnostics du modèle

La qualité du modèle est évaluée via ArviZ :

- $\hat{R}$ (**Gelman-Rubin**) : convergence des chaînes ($\hat{R} < 1.01$ requis) ;
- **ESS (Effective Sample Size)** : nombre effectif d'échantillons indépendants ;
- **Trace plots** : visualisation des chaînes MCMC ;
- R^2 **Bayésien** : qualité de l'ajustement ;
- **Posterior predictive checks** : comparaison prédictions vs. observations.

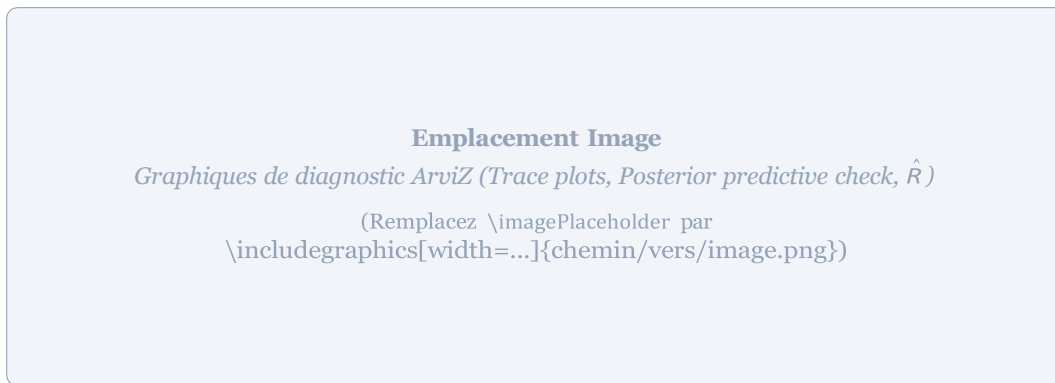


FIGURE 5 – Diagnostics du modèle Bayésien

4.6 Attribution multi-touch

À partir des coefficients postérieurs, la contribution de chaque canal est calculée par décomposition additive :

Contribution par canal

$$\text{Contribution}_c = \beta_c \cdot \frac{1}{T} \sum_{t=1}^T \tilde{x}_{c,t}^{\text{sat}} \quad (16)$$

La part de revenue attribuée au canal c est alors :

$$\text{Share}_c = \sum_c \frac{\text{Contribution}_c}{\text{Contribution}_c} \quad (17)$$

Les valeurs de Shapley (via le package shap) offrent une attribution optimisée respectant l'efficacité et la symétrie.

5 Référentiel des variables

5.1 Variable cible

Variable	Description
revenue	Prix total des premières achats — target principal du MMM
first_purchases	Nombre de premières acquisitions
avg_cpa	Coût par acquisition moyen

5.2 Canaux marketing

Canal	Code variable	Decay	Sat. s
Google Paid Search	google_search_spend	0.50	1.5
Google Shopping	google_shopping_spend	0.40	1.5
Google PMax	google_pmax_spend	0.60	1.5
Google Display	google_display_spend	0.70	1.2
Google Video	google_video_spend	0.65	1.3
Meta Facebook	meta_facebook_spend	0.60	1.4
Meta Instagram	meta_instagram_spend	0.65	1.3
Meta Other	meta_other_spend	0.55	1.4
TikTok	tiktok_spend	0.70	1.2

5.3 Variables de contrôle

Variable	Description
trend	Compteur temporel (jours depuis le début)
month_sin	$\sin(2\pi \cdot \text{mois}/12)$ — cyclicité saisonnière
month_cos	$\cos(2\pi \cdot \text{mois}/12)$ — cyclicité saisonnière
weekday	Jour de la semaine (0=lundi, 6=dimanche)
quarter	Trimestre (1–4)
is_critical_event	Flag événement critique (Black Friday, etc.)
is_high_impact	Flag événement fort impact
is_medium_impact	Flag événement impact moyen

TABLE 8 – Variables de contrôle utilisées dans le modèle

6 Visualisation et BI

Le projet propose deux couches de visualisation complémentaires : un dashboard **Streamlit** pour l'analyse exploratoire rapide, et des dashboards **Looker** pour le reporting professionnel partagé.

6.1 Architecture de visualisation

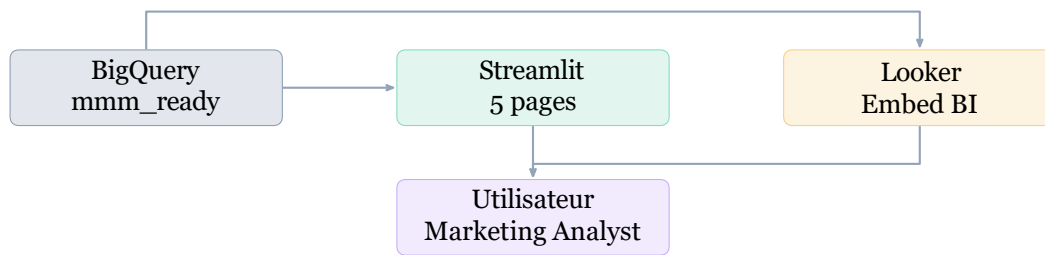


FIGURE 6 – Architecture de la couche visualisation

6.2 Dashboard Streamlit

L'application Streamlit (dashboard/app.py) comporte 5 pages :

Page	Icône	Fonctionnalités
Dashboard global		KPIs (Spend, Revenue, ROI), tendances, pie chart canal
Analyse des canaux		Sélecteur de canal, ROI, scatter Spend vs Revenue
Scénarios budgétaires		Sliders allocation %, validation budget, current vs proposed
Attribution		5 modèles (First/Last Touch, Linear, Time Decay, Shapley)
Looker / Config		Embed Looker, status données, config canaux

TABLE 9 – Pages du dashboard Streamlit

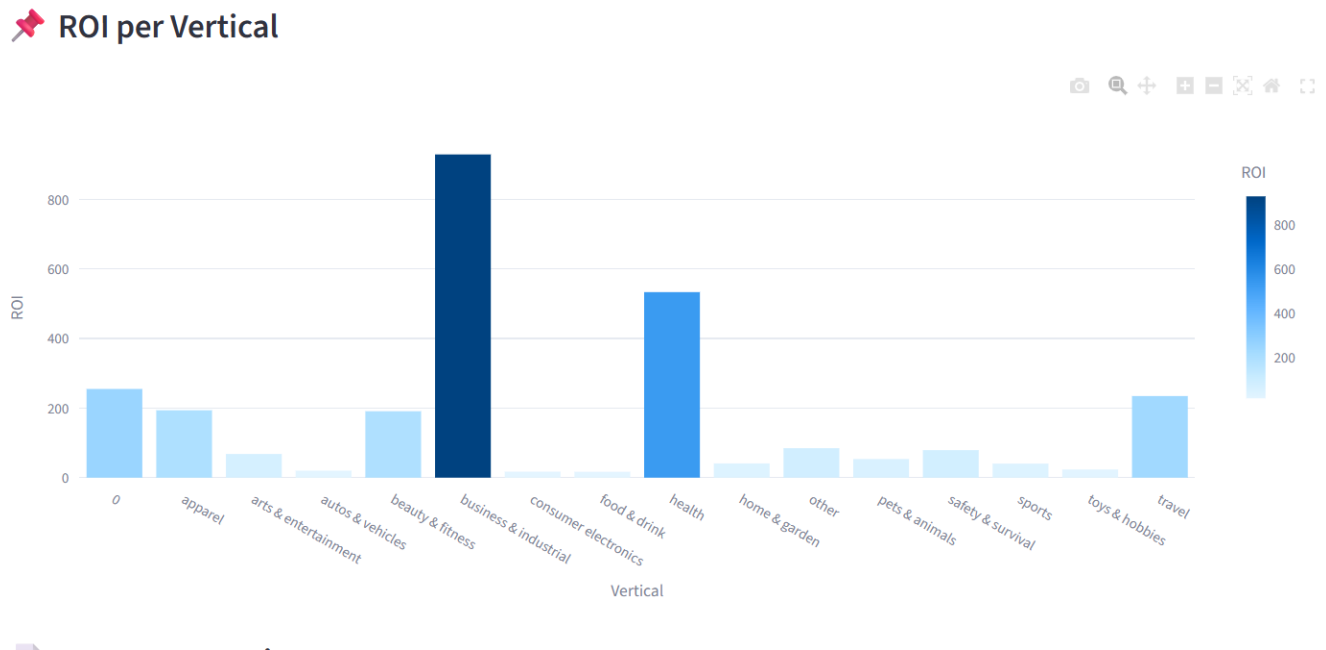


FIGURE 7 – Interface du dashboard Streamlit

6.2.1 Scénarios budgétaires (What-If)

La page Scénarios permet à l'analyste de :

1. visualiser l'allocation budgétaire actuelle ;
2. ajuster la part de chaque canal via des sliders ;
3. valider que la somme fait 100 % ;
4. comparer l'allocation proposée à l'actuelle ;
5. (à terme) obtenir une prédiction de revenue via le modèle Bayésien.

6.3 Intégration Looker

L'intégration Looker repose sur trois composants :

Fichier	Rôle
dashboard/looker_integration.py	Authentification API Looker, génération d'URL embed signées
dashboard/pages/looker_dashboards.py	Affichage des dashboards Looker en iframe dans Streamlit
looker/lookml_models.py	Templates LookML : vues, modèles, dashboards

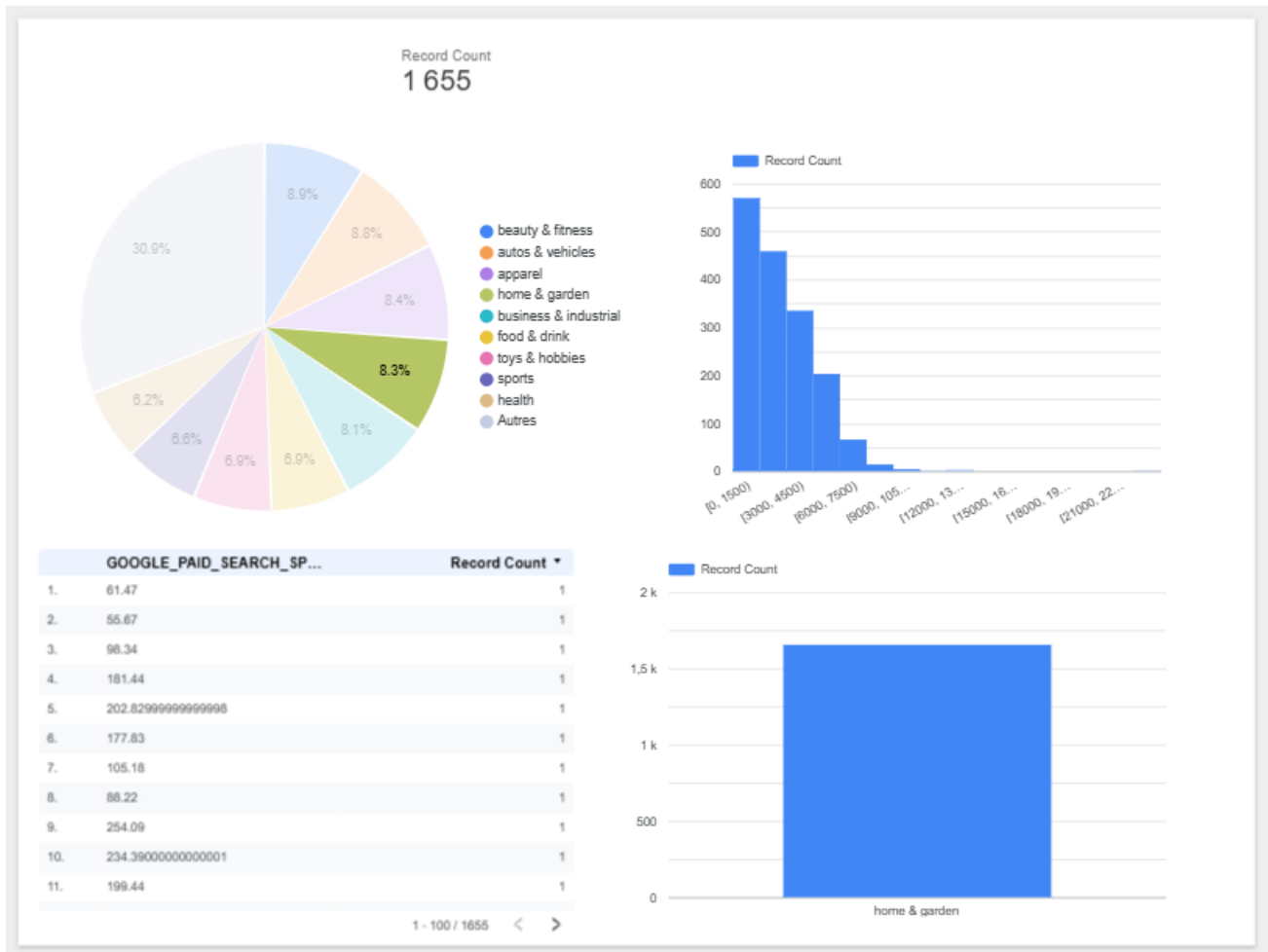


FIGURE 8 – Intégration Looker embarquée dans Streamlit

Configuration Looker requise

L'intégration Looker nécessite :

- un **Service Account** Looker avec permissions d'embed ;
- les variables `LOOKER_API_HOST`, `LOOKER_API_KEY`, `LOOKER_API_SECRET` configurées dans `.env` ;
- les dashboards déployés en production dans Looker.

7 Déploiement

7.1 Prérequis

- Python 3.10+
- Compte GCP avec BigQuery activé
- Instance Looker (optionnel)

7.2 Installation

Listing 1 – Installation et configuration

```
# 1. Cloner le dépôt
git clone <repo-url> && cd mmm-project

# 2. Installer les dépendances
pip install -r requirements.txt

# 3. Configurer les variables d'environnement
cp .env.example .env
# Editer .env avec vos valeurs
```

7.3 Variables de configuration

Variable	Description
GOOGLE_APPLICATION_CREDENTIALS	Chemin vers la clé JSON du service account GCP
GCP_PROJECT_ID	Identifiant du projet GCP
BIGQUERY_DATASET_ID	Dataset BigQuery cible
BIGQUERY_TABLE_ID	Table principale
RAW_DATA_PATH	Chemin vers les données brutes
PROCESSED_DATA_PATH	Répertoire de sortie
LOOKER_API_HOST	URL de l'instance Looker
LOOKER_API_KEY	Clé API Looker
LOOKER_API_SECRET	Secret API Looker
LOOKER_DASHBOARD_IDS	IDs des dashboards à embed

TABLE 10 – Variables d'environnement (.env)

7.4 Exécution

Listing 2 – Commandes d'exécution

```
# Pipeline ETL complete
python run_pipeline.py

# Options avancées
python run_pipeline.py --validate-only      # Validation seule
python run_pipeline.py --clean-only        # Nettoyage seul
python run_pipeline.py --bigquery          # Charger BQ apres pipeline
python run_pipeline.py --config            # Afficher la config

# Dashboard Streamlit
streamlit run dashboard/app.py
```

7.5 Artefacts de production

Fichier	Format	Usage
mmm_ready.csv	CSV	Données prêtes pour modélisation
adstock_params.pkl	Pickle	Paramètres de decay pour reproduction
normalization_params.pkl	Pickle	μ , σ pour dénormalisation
mmm_clean.csv	CSV	Snapshot après nettoyage
mmm_with_features.csv	CSV	Snapshot après feature engineering

TABLE 11 – Artefacts produits par la pipeline

8 Résultats attendus

Le système MMM complet doit permettre de :

1. **Produire un dataset analytique propre** — validation automatique, nettoyage intelligent, 52+ features transformées ;
2. **Calculer le ROI par canal** — contribution marginale et part de revenue attribuée ;
3. **Simuler des allocations budgétaires** — what-if analysis avec prédiction du modèle Bayésien ;
4. **Afficher des dashboards interactifs** — KPIs, tendances, scatter plots, attribution multi-touch ;
5. **Intégrer Looker** — dashboards embarqués, partage équipe, exports PDF.

8.1 Indicateurs de performance

KPI	Objectif
R^2 Bayésien	> 0.6 sur l'ensemble de test
$\hat{R}$ (Gelman-Rubin)	< 1.01 pour tous les paramètres
ESS (Effective Sample Size)	> 400 par chaîne
MAPE (Mean Abs. Perc. Error)	< 15 % sur les prédictions hors-échantillon
Couverture intervalle 95 %	≈ 95 % des observations dans l'intervalle prédictif

TABLE 12 – Indicateurs de performance du modèle

9 Perspectives et développements futurs

1. **Modélisation avancée** — priors hiérarchiques par vertical, modèle de mélange pour les événements, validation croisée temporelle ;
2. **Geo-experiments** — tests incrémentaux par région pour calibrer les coefficients du MMM ;
3. **Optimisation budgétaire** — algorithme d'optimisation sous contrainte (budget total, limites canal) ;
4. **Automatisation** — orchestration via Airflow/Prefect, GitHub Actions pour le CI/CD ;
5. **Monitoring** — alertes sur la dérive du modèle (concept drift), dashboards de suivi de performance ;
6. **Déploiement cloud** — Cloud Run / App Engine pour le dashboard, Vertex AI pour le modèle.

10 Conclusion

Ce projet fournit une base solide et industrialisable pour un système MMM complet. La pipeline Data — avec ses 5 étapes orchestrées, ses 45+ colonnes transformées et ses paramètres sauvegardés — est **prête pour la production**. Le cadre Bayésien défini permet d'estimer les contributions canal avec quantification des incertitudes. Les interfaces Streamlit et Looker offrent une double couche de visualisation, de l'exploratoire au reporting exécutif.

Les prochaines étapes — entraînement du modèle PyMC, attribution Shapley, déploiement automatisé — s'inscrivent naturellement dans cette architecture modulaire.

A Structure des répertoires

Listing 3 – Arborescence du projet

```
mmm - project/
|-- data/
|   |-- raw/
|   |   |-- compressed_data.csv
|   |   +-- reference/
|   |       +-- holidays.csv
|   +-- processed/
|       |-- mmm_clean.csv
|       |-- mmm_with_features.csv
|       |-- mmm_normalized.csv
|       |-- mmm_ready.csv
|       |-- adstock_params.pkl
|       +-- normalization_params.pkl
|-- etl/
|   |-- config.py
|   |-- validation.py
|   |-- clean.py
|   |-- event_enrichment.py
|   |-- feature_engineering.py
|   |-- normalize.py
|   |-- pipeline.py
|   +-- load_bigquery.py
|-- bigquery/
|   |-- schema.sql
|   +-- queries.sql
|-- models/
|-- dashboard/
|   |-- app.py
|   |-- lookout_integration.py
|   +-- pages/
|       +-- lookout_dashboards.py
|-- lookout/
|   +-- lookml_models.py
|-- run_pipeline.py
|-- requirements.txt
|-- .env.example
+-- README.md
```

B Exemple de code PyMC

Listing 4 – Structure du modèle Bayésien MMM avec PyMC

```
import pymc as pm
import numpy as np

with pm.Model() as mmm_model:
    # --- Priors ---
    intercept = pm.Normal("intercept", mu=0, sigma=2)
    beta_channels = pm.Normal("beta_channels",
                             mu=0, sigma=1, shape=9)
    beta_lags = pm.Normal("beta_lags",
                         mu=0, sigma=1, shape=27)
    beta_interact = pm.Normal("beta_interact",
                             mu=0, sigma=1, shape=2)
    beta_season = pm.Normal("beta_season", mu=0, sigma=1)
    beta_event = pm.Normal("beta_event", mu=0, sigma=1)
    beta_trend = pm.Normal("beta_trend", mu=0, sigma=1)
    sigma = pm.HalfNormal("sigma", sigma=1)

    # --- Likelihood ---
    mu = (intercept
          + pm.math.dot(beta_channels, X_channels.T)
          + pm.math.dot(beta_lags, X_lags.T)
          + pm.math.dot(beta_interact, X_interact.T)
          + beta_season * (month_sin + month_cos)
          + beta_event * is_critical_event
          + beta_trend * trend)
```

```
likelihood = pm.Normal("likelihood", mu=mu,
                      sigma=sigma, observed=y)

# --- Sampling ---
trace = pm.sample(2000, tune=1000, cores=4,
                  target_accept=0.95)
```

C Modèles LookML

Listing 5 – Extrait du template LookML (vue mmm)

```
view: mmm {
  sql_table_name: 'project.dataset.mmm' ;;

  dimension: date_day {
    type: date
    sql: ${TABLE}.date_day ;;
  }
  dimension: google_search_spend {
    type: number
    sql: ${TABLE}.google_search_spend ;;
  }
  dimension: meta_facebook_spend {
    type: number
    sql: ${TABLE}.meta_facebook_spend ;;
  }
  dimension: revenue {
    type: number
    sql: ${TABLE}.revenue ;;
  }

  measure: total_revenue {
    type: sum
    sql: ${revenue} ;;
  }
  measure: total_spend {
    type: sum
    sql: ${google_search_spend}
      + ${meta_facebook_spend} ;;
  }
  measure: roi {
    type: number
    sql: ${total_revenue} / NULLIF(${total_spend},0) ;;
    value_format: "0.00"
  }
}
```